# Optical Sudoku Solver Using Computer Vision and Machine Learning

Edward An
edward.an@ucalgary.ca

Karam Baroud
karam.baroud@ucalgary.ca

Mason Bullock
mason.bullock@ucalgary.ca

Koen Schoffner
koen.schoffner@ucalgary.ca

## Abstract

*We present an end-to-end optical Sudoku solver that combines classical computer vision techniques with machine learning-based digit recognition. The system processes an input image of a Sudoku puzzle, detects and rectifies the grid using perspective transformation, extracts individual cells, and classifies digits using a Support Vector Machine (SVM) trained on Histogram of Oriented Gradients (HOG) and Local Binary Pattern (LBP) features. To improve robustness, the system includes orientation invariance and adaptive thresholding for handling real-world lighting conditions. Experimental results demonstrate reliable performance on printed Sudoku puzzles under varying image conditions.*

## 1. Introduction

Sudoku is a widely recognized constraint-based puzzle that has become a common benchmark problem for evaluating computer vision pipelines involving detection, segmentation, and classification. Automatically solving Sudoku from images presents several challenges, including perspective distortion, noise, lighting variation, and digit recognition.

In this work, we develop a complete optical Sudoku solver that integrates image preprocessing, geometric transformation, and machine learning. Unlike end-to-end deep learning approaches, our method emphasizes interpretability and modularity by combining classical computer vision algorithms with a lightweight classifier. Custom functions are implemented for preprocessing steps such as Gaussian filtering, adaptive thresholding, polygon approximation algorithm, and perspective transformation, while leveraging OpenCV for contour detection and feature extraction. The digit recognition component is implemented using a Support Vector Machine (SVM) trained on a combination of Histogram of Oriented Gradients (HOG) and Local Binary Pattern (LBP) features, which are effective for capturing both shape and texture information.

## 2. Methodology

### 2.1. Initial Grid Detection Strategy

Initially, we explored using Harris Corner Detection to directly identify the intersections of the grid lines and locate the four outer corners of the board. However, this approach proved to be highly sensitive to noise, text within the cells, and background textures. The resulting corner response yielded numerous false positives, making it difficult to reliably isolate the main corners of the Sudoku board as shown in Figure 1.

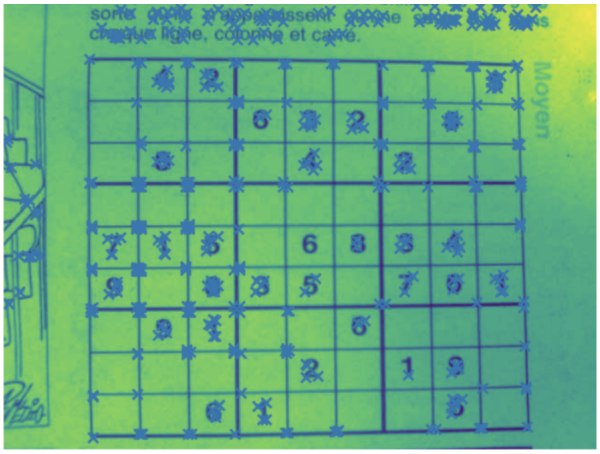

Figure 1. Harris corner detection response. The high number of false positives from text and noise made reliable grid extraction unfeasible.

Due to this unreliability, we pivoted to an edge-based topological approach, utilizing contour detection paired with a polygon approximation algorithm, which provided a much more robust extraction of the grid's outer boundary.

## 2.2. Preprocessing and Grid Detection

The input image is first converted to grayscale and smoothed using a Gaussian filter to reduce noise. We implement Gaussian filtering manually using a generated kernel and convolution operation.

Adaptive thresholding is applied to convert the image into a binary representation. Unlike global thresholding, adaptive thresholding computes a local threshold for each pixel, allowing the system to handle uneven lighting conditions. Here is a comparison between global and adaptive thresholding (see Figure 2):

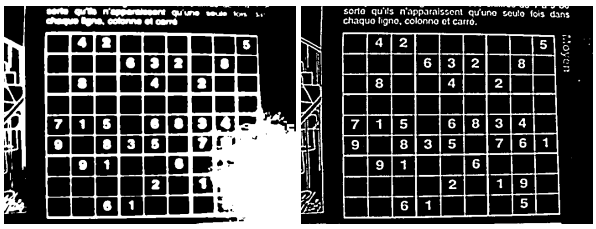

Figure 2. Comparison of Otsu global thresholding (left) and adaptive thresholding (right). Adaptive thresholding provides better results under non-uniform lighting conditions.

The resulting binary image is then processed to detect contours.

Contours are extracted using OpenCV's `findContours` function. This function returns a list of contours found in the binary image. Each contour is given in the format of `[[x_1, y_1], [x_2, y_2], ..., [x_n, y_n]]`, where $(x_i, y_i)$ are the coordinates of the contour points. The area of each contour is calculated using the Shoelace Formula, which computes the area based on the vertices of the polygon formed by the contour points. The contour with the largest area that approximates a quadrilateral is assumed to correspond to the Sudoku grid (see Figure 3).

To identify the corners of the grid, we apply a polygon approximation algorithm (Ramer-Douglas-Peucker) to simplify the contour from n points to four points. These four corner points are then used for perspective transformation to obtain a top-down view of the Sudoku board, which is essential for accurate cell extraction and digit recognition.

## 2.3. Perspective Transformation

Once the grid corners are detected, a perspective transformation is applied to obtain a top-down view of the Sudoku board. The board is warped into a fixed square resolution (450x450), enabling consistent downstream processing. The perspective transformation is computed using the four detected corner points and the corresponding points in the output image. The transformation matrix is calculated using OpenCV's `getPerspectiveTransform` function, and the image is warped using `warpPerspective`.

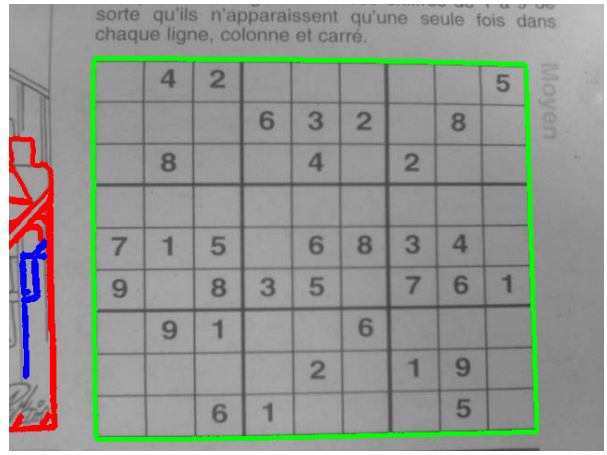


Figure 3. Contour detection result showing the three largest contours by area. The green outline represents the largest quadrilateral contour identified as the Sudoku grid.

In order to prevent holes in the warped image, we use inverse perspective warping. This approach ensures that every pixel in the output image is assigned a value based on the corresponding location in the input image, avoiding gaps that can occur with forward mapping.

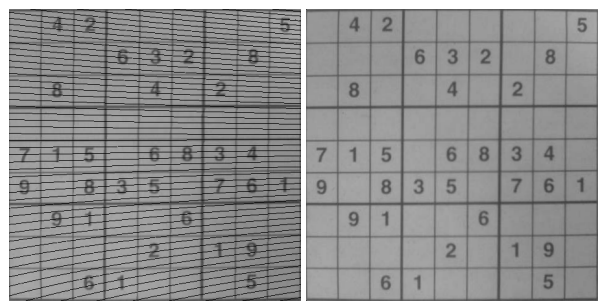

Figure 4. Perspective transformation result showing the forward warped image (left) and the inverse warped top-down view of the Sudoku board (right).

## 2.4. Cell Extraction

The warped square board is divided into a $9 \times 9$ grid, producing 81 individual cells. Each cell is cropped with margins to remove grid lines (see Figure 5).

To determine whether a cell is empty, we compute the ratio of foreground (ink) pixels using adaptive thresholding. Cells with low ink density are classified as empty and skipped during digit recognition.

## 2.5. Orientation Invariance

To handle images captured at arbitrary rotations, the system evaluates four orientations (0°, 90°, 180°, 270°). For each orientation, digit classification confidence scores are

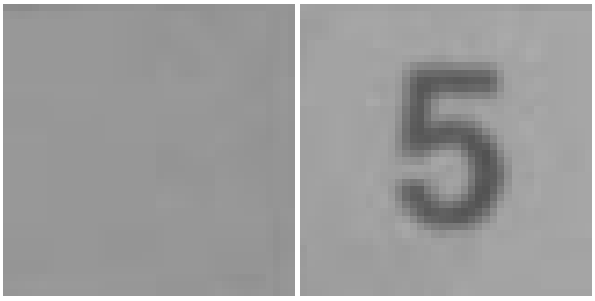

Figure 5. Example of extracted cells from the warped Sudoku board. The left image shows the top-left cell, while the right image shows the top-right cell. Margins are applied to avoid grid lines.

computed using the `decision_function` of the trained model. The score represents the distance and position of input data point relative to the hyperplane created in the training phase. The orientations are sorted in descending order of average confidence. The system attempts to solve the Sudoku puzzle starting with the highest confidence orientation. If a valid solution is found, it is returned; otherwise, the next orientation is evaluated until a solution is obtained or all orientations are exhausted.

### 2.6. Digit Recognition

Digit classification is performed using a Support Vector Machine (SVM). Each cell is normalized to a fixed size and centered within a canvas to ensure consistency.

We extract two complementary feature types:

- **HOG features:** capture gradient structure and digit shape
- **LBP features:** capture local texture and topology

The combined feature vector improves discrimination between visually similar digits such as 3, 6, 8, and 9.

### 2.7. Sudoku Solving

The digits recognized from the cells are used to populate a partially filled Sudoku board. This board is validated and solved using a backtracking algorithm. The final solution is displayed if a valid configuration is found (see Figure 6).

## 3. Results

The proposed optical Sudoku solver demonstrated highly reliable performance under ideal to moderate real-world conditions. The integration of adaptive thresholding and inverse perspective warping proved highly effective in standardizing the grid against non-uniform lighting and severe viewing angles. Furthermore, the orientation-invariant classification mechanism successfully resolved puzzles captured at arbitrary rotations.

```
Unsolved board:
. 4 2 | . . . | . . 5
. . . | 6 3 2 | . 8 .
. 8 . | . 4 . | 2 . .
------+-------+------
. . . | . . . | . . .
7 1 5 | . 6 8 | 3 4 .
9 . 8 | 3 5 . | 7 6 1
------+-------+------
. 9 1 | . . 6 | . . .
. . . | . 2 . | 1 9 .
. . 6 | 1 . . | . 5 .
```

```
Solved board:
3 4 2 | 8 9 1 | 6 7 5
1 5 7 | 6 3 2 | 9 8 4
6 8 9 | 7 4 5 | 2 1 3
------+-------+------
4 6 3 | 9 1 7 | 5 2 8
7 1 5 | 2 6 8 | 3 4 9
9 2 8 | 3 5 4 | 7 6 1
------+-------+------
5 9 1 | 4 7 6 | 8 3 2
8 7 4 | 5 2 3 | 1 9 6
2 3 6 | 1 8 9 | 4 5 7
```

Figure 6. The left image shows the unsolved Sudoku board with recognized digits, while the right image shows the solved board after applying the backtracking algorithm.

The SVM classifier, trained on a composite of HOG and LBP features, achieved strong discrimination between visually similar digits (e.g., 3, 6, 8, and 9) across both synthetic and real-world datasets. However, as detailed in the Discussion, the system's end-to-end success rate is heavily bottlenecked by the quality of the initial contour detection. Performance degrades when confronted with extreme blur, occluded grid boundaries, or significant non-linear page distortion, as these environmental factors cause cascading failures in the cell extraction and digit recognition phases.

## 4. Discussion

While the proposed system performs reliably end-to-end under ideal conditions, real-world testing revealed several distinct limitations and edge cases:

- **Information Loss in Blurry Images:** The Suzuki-Abe topological structural analysis algorithm (used by OpenCV's contour detection) relies heavily on continuous binary lines. Blurry images suffer from information loss, leading to fragmented lines after thresholding. We attempted to mitigate this by calculating the variance of the Laplacian to estimate a blur score, bypassing the Gaussian blur step if the score fell below a certain threshold. We also experimented with morphological operations (closing and dilation) to reconnect broken lines. However, while this fixed some blurry cases, it degraded the performance on clear images, making it an unreliable global solution.
- **Partial Grid Visibility:** Because the contour detection requires a closed boundary to identify the board, the algorithm fails if the outer edge of the Sudoku grid is cut off by the camera frame. An unbroken quadrilateral cannot be formed, causing the pipeline to fail early (see Figure 7).
- **Background Contour Interference:** The pipeline

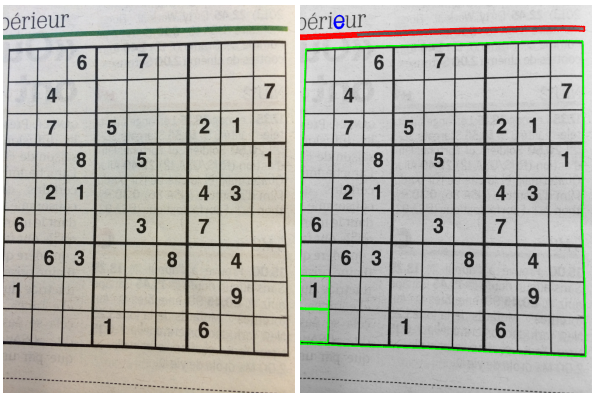

Figure 7. Failure case due to partial grid visibility. The left edge is cut off from the image frame, preventing closed contour detection. The right image shows the failed contour detection result, where no valid quadrilateral is identified.

currently operates on the assumption that the Sudoku board will be the largest contour in the field of view. If a larger, closed geometric shape is present in the background, the algorithm will incorrectly target it instead of the puzzle (see Figure 8). This issue is relatively minor in practice, as the user can easily adjust the camera distance to ensure the board dominates the frame.

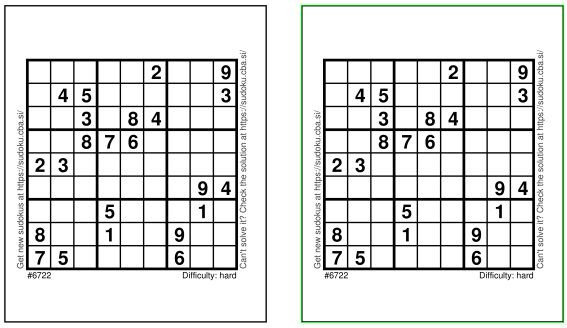

Figure 8. Failure case due to background contour interference. A larger shape in the background is incorrectly identified as the Sudoku board (see green outline on the right image).

- **Non-linear Distortions:** Perspective transformation assumes the original surface is planar. If the paper itself is bent or curved excessively, warping the four outer corners to a fixed $450 \times 450$ square does not straighten the inner grid lines. Consequently, our assumption that each cell is geometrically uniform fails; curved grid lines encroach into the extracted cell margins. The `is_cell_empty` function detects this grid ink, falsely classifying the cell as populated. When this noisy cell is passed to the SVM, it results in false digit predictions for cells that were actually empty.

Despite these limitations, the modular design of the pipeline allows individual components to be improved independently. For example, replacing the SVM with a Convolutional Neural Network (CNN) could yield higher accuracy, and implementing a grid-line intersection calculation could help map bent, non-linear grids more accurately than a planar perspective warp.

## 5. Conclusion

In this paper, we presented a complete, modular optical Sudoku solver combining classical computer vision pipelines with machine learning. By utilizing contour detection and polygon approximation, we successfully extracted Sudoku grids from unstructured images. Applying inverse perspective warping allowed us to geometrically normalize the puzzle cells for a lightweight SVM classifier, which leveraged robust shape and texture features for accurate digit recognition.

While the system effectively navigates common challenges like perspective distortion and lighting variations, our testing highlighted the limitations of classical topological algorithms when faced with fragmented lines, partial framing, and non-planar physical surfaces. Demonstrating the viability of hybrid approaches for structured visual tasks, this work also underscores the necessity for resilient error-handling in early preprocessing stages.

Future work will focus on addressing these specific edge cases by implementing grid-line intersection mapping for bent pages, as well as replacing the SVM with a Convolutional Neural Network (CNN) to expand system compatibility to handwritten puzzles and highly degraded digits.